

Modelos para MVP técnico

En esta conversación me hablaste sobre estos tres modelos:

Para un MVP serio:

Transformer multimodal BEV para percepción

Planificación híbrida: ML + planner clásico + MPC

Pose estimation para aperos, pero en segunda fase si quieres acortar plazos

Necesito ahora una explicación técnica y detallada de cada punto. Incluye en la explicación qué modelo de machine learning se emplearía para cada uno (supervised learning, unsupervised, Deep Learning, Reinforcement Learning...)

RouteLLM

He revisado la conversación del PDF y además he hecho la búsqueda web que pediste.

Tus tres puntos están bien elegidos para una arquitectura seria de autonomía móvil. De hecho, encajan con la literatura reciente: los *transformers* multimodales dominan percepción/fusión, la planificación robusta sigue siendo mejor como sistema híbrido y la estimación de pose 6D sigue siendo la vía natural para tareas de acople/manipulación fina [ScienceDirect](#), [OpenReview](#), [MathWorks](#), [GitHub 6D Pose Estimation](#).

Resumen corto antes del detalle

Punto	Qué resuelve	Tipo principal
Transformer multimodal BEV para percepción	Entender el entorno a partir de cámara/LiDAR/radar/estado ego	Deep Learning supervisado
Planificación híbrida: ML + planner clásico + MPC	Decidir una trayectoria segura, factible y controlable	Mixto: ML supervisado / imitation learning + algoritmos clásicos + control óptimo
Pose estimation para aperos	Saber con precisión dónde está el apero y cómo alinearlos	Deep Learning supervisado + geometría 3D

Ahora voy punto por punto, a nivel técnico.

1) Transformer multimodal BEV para percepción

Qué es exactamente

Aquí la idea es construir un modelo de percepción que tome sensores heterogéneos:

- cámaras RGB
- LiDAR
- radar
- IMU/GNSS/odometría como contexto ego

y los convierta en una representación común en **BEV** (*Bird's-Eye View*), es decir, una vista cenital del entorno.

Esa representación BEV es muy potente porque:

- la planificación trabaja naturalmente en coordenadas top-down,
- la geometría del vehículo y del apuro se expresa mejor en planta,
- es más fácil razonar sobre:
 - ocupación
 - espacio libre
 - obstáculos
 - pasillos
 - líneas de cultivo
 - zonas no transitables.

La revisión reciente sobre *transformers* en conducción autónoma destaca precisamente su utilidad en percepción, fusión sensorial, predicción y *motion*

planning, resaltando ventajas frente a CNN/RNN en contexto global y alineación multimodal [ScienceDirect](#).

Arquitectura típica

Una implementación razonable sería tipo **BEVFusion / BEVFormer-like**:

1.

Encoder por sensor

- cámara: backbone CNN o ViT para extraer *features* 2D
- LiDAR: voxelization / pillarization + backbone 3D o sparse conv
- radar: representación rasterizada o puntos + encoder específico

2.

Proyección a marco común

- las *features* de cámara se proyectan al espacio 3D/BEV usando calibración extrínseca e intrínseca
- las de LiDAR/radar ya están cerca del marco geométrico del mundo

3.

Fusión multimodal

- mediante **cross-attention** entre sensores
- opcionalmente también **temporal attention** para usar historial

4.

Heads de salida

- detección 3D de objetos
- segmentación semántica BEV
- occupancy / free-space

- estimación de transitabilidad
- predicción de objetos dinámicos

Qué salidas interesa producir en tu caso

Para maquinaria agrícola autónoma, yo pediría al modelo estas salidas:

- **occupancy grid** o mapa de ocupación
- **free-space map**
- **detección 3D** de:
 - árboles
 - personas
 - animales
 - vehículos
 - remolques
 - postes
 - maquinaria
- **segmentación de suelo**
 - transitable
 - no transitable
 - vegetación alta
 - cultivo
 - borde de camino
 - cabecera

- **incertidumbre/confianza**
- opcional:
 - *drivable corridor*
 - estimación de borde de surco o línea de trabajo.

Por qué BEV es especialmente adecuado aquí

Porque tu problema no es “ver bonito”, sino **operar geometría**:

- ancho de máquina
- ancho de apero
- radio de giro
- barrido trasero
- distancia de seguridad
- ocupación efectiva del conjunto tractor + implemento.

En BEV todo eso se modela mucho mejor que en imagen frontal pura.

Qué tipo de machine learning se emplea

Aquí la respuesta principal es:

- **Deep Learning**
- normalmente **Supervised Learning**

y secundariamente:

- **Self-Supervised Learning** o *pretraining* multimodal, si quieres mejorar robustez con mucho dato no etiquetado

- algo de **Semi-Supervised Learning**, si el etiquetado completo es caro.

Pero el núcleo productivo suele ser **supervisado**.

Cómo se entrena

Con datos sincronizados y calibrados, y etiquetas como:

- cajas 3D
- máscaras BEV
- ocupación
- clases semánticas
- trayectorias de objetos

Pérdidas típicas:

- clasificación: focal loss / cross entropy
- regresión de cajas: L1 / smooth L1 / IoU-like
- segmentación: CE + Dice
- occupancy: BCE / focal
- temporal consistency loss si metes memoria

Ventajas técnicas

- captura contexto espacial global
- fusiona sensores muy distintos
- tolera mejor oclusiones parciales
- produce un mapa topológico útil para planificación

- puede incorporar historia temporal

Inconvenientes reales

- requiere **sincronización** y **calibración** muy finas
- coste computacional alto
- BEV denso puede penalizar FPS y memoria; eso también lo comenta trabajo reciente sobre arquitecturas end-to-end con BEV denso [OpenReview](#)
- en campo tendrás problemas de:
 - polvo
 - vibración
 - suciedad en cámara/LiDAR
 - GNSS degradado bajo arbolado.

Mi recomendación concreta

Para tu caso, este es el **modelo más importante** del sistema.

Si hay que priorizar, este va primero.

Tipo ML resumido

- **Principal**: Supervised Deep Learning
- **Secundario útil**: Self-supervised pretraining / semi-supervised
- **No principal**: Reinforcement Learning

2) Planificación híbrida: ML + planner clásico + MPC

Este punto es muy importante entenderlo bien: **no es un solo modelo ML**.

Es un **sistema híbrido** donde cada parte hace algo distinto.

Qué problema resuelve

Dado:

- un mapa BEV / occupancy
- estado del vehículo
- objetivo o misión
- restricciones geométricas y dinámicas

quieres obtener una trayectoria que sea a la vez:

- segura
- factible cinemáticamente
- eficiente
- suave
- ejecutable por el controlador.

Por qué no recomiendo “solo ML”

Porque planificación en maquinaria pesada tiene restricciones duras:

- no colisionar
- respetar radio de giro
- respetar límites de dirección
- controlar derrape/patinamiento
- no invadir zonas prohibidas
- no generar maniobras imposibles con remolque o apero.

Un modelo puro aprendido puede proponer trayectorias “parecidas” a las humanas, pero fuera de distribución puede hacer cosas absurdas.

Por eso la mejor arquitectura práctica es:

1. **ML propone o puntúa**
2. **planner clásico verifica/genera candidatos**
3. **MPC ejecuta respetando restricciones**

Descomposición técnica correcta

A. Capa global

Se encarga de:

- cobertura de parcela
- secuencia de pasadas
- orden de tareas
- waypoints globales
- minimizar solapamientos y maniobras inútiles

Aquí normalmente **no usaría ML como núcleo**.

Usaría algoritmos clásicos de:

- *coverage path planning*
- optimización sobre grafo
- descomposición tipo boustrophedon
- heurísticas con coste operativo

Esto no es Machine Learning. Es **optimización clásica / planificación algorítmica**.

B. Capa local

Se encarga de:

- esquivar obstáculos
- recalcular a corto horizonte
- elegir entre varias maniobras
- decidir si seguir, frenar o parar seguro

Aquí sí tiene sentido meter una parte aprendida.

Ejemplos:

- red que genere trayectorias candidatas
- red que puntúe trayectorias
- *learned cost map*
- *trajectory transformer*
- *diffusion policy* para maniobras cortas
- *imitation learning* a partir de conducción experta

C. Capa de validación física/seguridad

Esta capa comprueba:

- colisión
- límites cinemáticos
- geofence
- viabilidad con el apuro
- márgenes de seguridad

- reversa o maniobra admisible

Aquí usaría:

- **Hybrid A***
- **lattice planner**
- *collision checker*
- restricciones geométricas del vehículo-articulación

D. Control de seguimiento

Una vez elegida la trayectoria, toca ejecutarla.

Aquí entra **MPC**.

MathWorks resume bien el rol de MPC: resolver un problema de optimización sobre un horizonte, con restricciones explícitas, tanto lineales como no lineales, aplicándolo a planificación de rutas, seguimiento y otras tareas de conducción autónoma [MathWorks](#).

Qué hace el MPC exactamente

El **Model Predictive Control** usa un modelo del vehículo para predecir su evolución futura y resolver, en cada ciclo, una optimización:

- minimiza error respecto a trayectoria
- penaliza cambios bruscos de control
- respeta restricciones:
 - dirección
 - velocidad
 - aceleración
 - curvatura

- estabilidad
- distancia a obstáculos
- límites del actuador

Conceptualmente, resuelve algo tipo:

$$\min_{u_{0:H-1}} \sum_{k=0}^{H-1} \left(\|x_k - x_k^{ref}\|_Q^2 + \|u_k\|_R^2 + \|\Delta u_k\|_S^2 \right)$$

sujeto a

$$x_{k+1} = f(x_k, u_k)$$

y restricciones

$$u_{min} \leq u_k \leq u_{max}, \quad x_k \in \mathcal{X}_{safe}$$

Ojo importante: MPC no es ML

Esto conviene dejarlo nítido.

- **MPC** no es *machine learning*
- es **control óptimo / optimización online basada en modelo**

Puede coexistir con ML, pero no pertenece a supervised/unsupervised/RL.

Entonces, ¿qué “modelo de ML” hay aquí?

Depende de qué parte híbrida quieras aprender.

La opción más sólida para un MVP serio es:

- **ML para propuesta o scoring de trayectorias**
- **clásico para seguridad y factibilidad**

- **MPC para ejecución**

El ML aquí suele ser:

- **Supervised Learning** mediante **Imitation Learning**
- implementado con **Deep Learning**

Ejemplos de entrenamiento:

- entrada:
 - occupancy map
 - estado ego
 - objetivo local
 - historial reciente
- salida:
 - trayectoria futura
 - distribución sobre maniobras
 - ranking de candidatos

¿Se puede usar Reinforcement Learning?

Sí, pero yo no lo pondría como primera opción en producción.

RL puede servir para:

- afinar maniobras
- optimizar eficiencia
- aprender estrategias en simulación

- *fine-tuning* local

Pero en sistemas reales de maquinaria pesada tiene pegadas:

- sample inefficiency
- difícil garantizar seguridad
- sim2real complicado
- comportamiento impredecible fuera del dominio.

Mi criterio:

RL como herramienta secundaria, no como núcleo del MVP.

Diseño que yo recomendaría

- **Global planner**: clásico
- **Local planner**: red supervisada o imitation learning para generar/scorar maniobras
- **Safety layer**: clásica, obligatoria
- **Trajectory tracking**: NMPC o MPC

Tipo ML resumido

Este punto es mixto:

- **ML principal**:
 - **Supervised Learning**
 - **Imitation Learning**
 - implementado con **Deep Learning**

- **Opcional:**
 - **Reinforcement Learning** en simulación
- **No ML pero crítico:**
 - planner clásico
 - Hybrid A*
 - lattice
 - collision checker
 - MPC

Mi opinión técnica

Este segundo punto no debe entenderse como “un modelo”, sino como **la arquitectura decisional completa**.

Y, en producto real, suele ser la solución correcta.

3) Pose estimation para aperos

Qué problema resuelve exactamente

Aquí ya no estás en navegación general, sino en **manipulación geométrica fina**.

Ejemplos:

- alinear tractor con remolque
- alinear cosechadora con peine
- aproximar enganche a bulón/punto de acople
- saber orientación relativa apero-vehículo

- controlar error lateral, longitudinal y angular en maniobra lenta.

Esto exige estimar con precisión la pose relativa del apero respecto al vehículo o cámara.

Qué significa pose estimation aquí

En términos técnicos, suele ser **estimación de pose 6D**:

- 3 componentes de traslación
- 3 de rotación

Es decir, calcular la transformación rígida:

$$T = [R, t]$$

que lleva del sistema de coordenadas del apero al de la cámara o al vehículo.

Esa es precisamente la definición estándar de **6D object pose estimation** [GitHub 6D Pose Estimation](#).

Arquitectura práctica para aperos

La pipeline típica sería:

1. **detección del apero / punto de enganche**
2. **segmentación o keypoints**
3. **estimación de pose 6D**
4. **refinamiento geométrico**
5. **visual servoing / control fino**

Enfoques que sí encajan

Opción 1: Keypoint detection + geometría

La red detecta puntos clave:

- boca de enganche
- bulón
- extremos del apero
- referencias visuales rígidas

Luego, con la geometría conocida del apero, resuelves pose con PnP o equivalente.

Esto es muy útil cuando:

- tienes aperos conocidos
- puedes definir puntos semánticos claros
- quieres interpretabilidad

Opción 2: Instance segmentation + pose regression

La red primero segmenta el apero y luego estima directamente:

- traslación
- orientación

Puede funcionar bien cuando la forma visual es bastante consistente.

Opción 3: RGB-D / point cloud 6D pose

Si montas cámara de profundidad o LiDAR cercano, puedes estimar pose sobre nube de puntos.

La recopilación de métodos de 6D pose muestra justo esa división entre métodos basados en:

- **RGB-D**

- **point cloud**
- **semantic keypoints**
- **segmentation-driven**
- **point-based networks** [GitHub 6D Pose Estimation](#)

Qué sensores usaría

Para esta tarea concreta, yo usaría:

- cámara cercana trasera o lateral trasera
- idealmente RGB-D o estéreo si el presupuesto lo permite
- iluminación controlada si es posible
- IMU/odometría como apoyo
- opcional fiducials si quieres acelerar MVP

Qué tipo de machine learning se emplea

Aquí el tipo principal vuelve a ser:

- **Supervised Learning**
- dentro de **Deep Learning**

Modelos típicos:

- CNNs de keypoints
- redes de segmentación
- redes de regresión de pose 6DoF
- PointNet/Point Transformer-like si usas nube de puntos
- refinamiento iterativo

También es muy útil:

- **Self-Supervised / Weakly Supervised** para aprovechar sintético + pocas anotaciones reales
- *domain randomization* en simulación

Qué no usaría como núcleo

- **Unsupervised puro**: no suele ser suficiente para precisión industrial
- **RL como núcleo de pose**: no; RL sería para la política de maniobra, no para estimar pose

Salidas necesarias

No basta con “detectar el apero”.

Necesitas al menos:

- posición relativa x, y, z
- orientación relativa ($roll, pitch, yaw$) o representación equivalente
- confianza de estimación
- visibilidad/oclusiones
- error estimado de alineación

Por qué esto no iría en el MVP inicial

Porque exige:

- dataset muy específico
- instrumentación y etiquetado fino
- maniobras lentas y seguras

- tolerancias pequeñas
- integración estrecha con control

Además, la suciedad, vibración, barro y oclusiones te afectan muchísimo más aquí que en navegación abierta.

Por eso tiene sentido dejarlo para **fase 2 o 3** si quieres acortar plazos.

Mi recomendación concreta

Si lo haces, yo iría por:

- **detección/segmentación del apuro**
- **keypoints semánticos**
- **estimación de pose 6D**
- **refinamiento geométrico**
- **control visual lento con safety layer**

Tipo ML resumido

- **Principal:** Supervised Deep Learning
- **Complementario:** Self-supervised / synthetic-to-real
- **No principal:** RL
- **Imprescindible pero no ML:** geometría 3D, PnP/ICP, visual servoing, control

Conclusión técnica comparativa

Punto 1: Transformer multimodal BEV para percepción

Es el **modelo de percepción principal**.

Sirve para convertir sensores brutos en una representación estructurada del entorno.

- Tipo: **Deep Learning supervisado**
- Papel: percepción y fusión
- Prioridad: **máxima**

Punto 2: Planificación híbrida ML + planner clásico + MPC

Es la **arquitectura de decisión y control**, no un modelo único.

- ML: **supervisado / imitation learning / deep learning**
- Planner clásico: **no ML**
- MPC: **control óptimo, no ML**
- Prioridad: **máxima**, junto con percepción

Punto 3: Pose estimation para aperos

Es un bloque especializado para **acople y maniobra fina**.

- Tipo: **Deep Learning supervisado**
- Apoyo: geometría 3D y control visual
- Prioridad: **fase posterior** salvo que el negocio dependa desde el día 1 del docking automático

Mi recomendación final, sin rodeos

Para un MVP realista y serio:

1. **Transformer multimodal BEV para percepción**

porque necesitas entender el entorno en una representación útil para planificar.

2.

Planificación híbrida: learned local planning + planner clásico + MPC

porque es la forma más robusta de tomar decisiones seguras y ejecutables.

3.

Pose estimation para aperos en segunda fase

porque aporta mucho valor, pero aumenta bastante el riesgo de calendario y de dataset.

Si quieres, en el siguiente mensaje te preparo una versión aún más técnica con uno de estos dos formatos:

1.

Tabla de arquitectura completa

- módulo
- input
- output
- algoritmo/modelo
- tipo de aprendizaje
- métricas

2.

Diseño de stack MVP

- sensores
- topics ROS2
- modelos concretos
- pipeline de entrenamiento
- latencias objetivo
- roadmap de validación.

Eres estudiante de un máster en vehículos autónomos y conectados. Tienes que explicarle a tu tutor del TFM qué quieres hacer.

Resume los tres puntos de manera concisa, clara y directa. Sé técnico y claro. Incluye justificación, modelos, stack tecnológico propuesto...

Desarrollalo teniendo en cuenta lo siguiente: Objetivo tener el BEV y la planificación híbrida. El punto 3, pose estimation, se queda en líneas futuras de desarrollo

RouteLLM

Propuesta de TFM explicada al tutor

Mi propuesta de TFM es desarrollar un MVP de autonomía vehicular centrado en dos bloques principales:

1. Percepción del entorno en BEV mediante un modelo multimodal basado en Transformer
2. Planificación híbrida de trayectorias: percepción + planner clásico + MPC

El tercer punto, pose estimation para aperos, lo dejaría como línea futura de desarrollo, no como alcance del TFM.

1) Percepción: Transformer multimodal en BEV

Lo primero que quiero construir es una **representación BEV (*Bird's-Eye View*) del entorno** a partir de varios sensores, principalmente:

- cámaras RGB
- LiDAR
- GNSS/IMU/odometría

La idea es fusionar esa información en una **vista cenital métrica** que me permita saber:

- qué espacio está libre
- qué espacio está ocupado
- dónde están los obstáculos
- qué zonas son transitables
- cuál es la geometría útil para planificar

Justificación técnica

La BEV es una representación muy adecuada para autonomía porque convierte el entorno en un plano donde la planificación y el control tienen sentido físico directo. En la literatura reciente se ve que la representación BEV mejora tareas como percepción, segmentación del espacio transitable, localización y planificación. Por ejemplo, BEV-ODOM justifica que la representación BEV aporta **consistencia de escala** y reduce deriva en estimación de movimiento; JPerceiver muestra que profundidad, pose y layout BEV pueden aprenderse conjuntamente; y BEVMapMatch destaca que el BEV es especialmente útil para **map understanding, drivable-region segmentation y localización robusta**.

Modelo que usaría

Aquí el núcleo sí sería claramente **Deep Learning supervisado**:

- **Transformer multimodal**
- aprendizaje principal: **Supervised Learning**
- apoyo posible: **self-supervised pretraining** o **semi-supervised learning**

Arquitectura propuesta

Una arquitectura razonable sería:

- **encoder por sensor**
 - cámara: backbone CNN o ViT

- LiDAR: voxel/pillar encoder
- **proyección a espacio común**
- **fusión multimodal**
 - cross-attention
 - temporal attention si se usa historial
- **heads de salida**
 - occupancy grid
 - free-space map
 - segmentación semántica BEV
 - detección de obstáculos
 - opcionalmente estimación de incertidumbre

Resultado esperado

Este bloque me debería entregar un **mapa estructurado del entorno** que alimente la planificación.

En otras palabras: el sistema no “ve imágenes”, sino que “entiende el espacio operativo”.

2) Planificación híbrida: ML + planner clásico + MPC

El segundo gran bloque del TFM sería la **planificación híbrida**.

Aquí el objetivo no es solo detectar el entorno, sino **decidir una trayectoria segura, suave y físicamente ejecutable**.

Idea principal

No quiero hacer una planificación puramente basada en red neuronal, porque en un sistema real eso es frágil.

La propuesta es una arquitectura híbrida donde:

- **la percepción BEV** aporta el estado del entorno
- un **planner clásico** garantiza factibilidad y seguridad
- un bloque aprendido puede ayudar a **proponer o puntuar trayectorias**
- un **MPC** ejecuta la trayectoria respetando restricciones dinámicas

Justificación técnica

Esta arquitectura es la más seria para un MVP real porque combina:

- robustez
- interpretabilidad
- restricciones físicas explícitas
- capacidad de aprovechar ML sin depender completamente de él

Además, en problemas de navegación real, el BEV encaja de forma natural con pose 3-DoF y con planificación local sobre mapa ocupacional. La idea de usar BEV como espacio intermedio para tareas de localización y razonamiento geométrico aparece también en trabajos como C-BEV, BEV-ODOM y BEVMapMatch.

Cómo dividiría esta planificación

Capa global

Se encarga de:

- waypoints

- ruta general
- cobertura o secuencia de trayectorias
- objetivo global

Aquí usaría **algoritmos clásicos**, no ML.

Capa local

Se encarga de:

- esquivar obstáculos
- replanificar a corto horizonte
- elegir maniobra local

Aquí sí puede entrar ML, por ejemplo:

- red que puntúe trayectorias
- red que genere trayectorias candidatas
- learned cost map
- imitation learning sobre conducción experta

Capa de validación

Valida:

- colisión
- geofence
- radio de giro
- restricciones cinemáticas

Aquí usaría de nuevo métodos clásicos como:

- lattice planner
- Hybrid A*
- collision checker

Capa de control

Aquí usaría **MPC** para seguir la trayectoria.

El MPC me interesa porque permite optimizar online bajo restricciones explícitas:

- velocidad
- dirección
- aceleración
- curvatura
- seguimiento de referencia

Tipo de ML en este bloque

Este punto es **mixto**:

- parte aprendida:
 - **Supervised Learning**
 - **Imitation Learning**
 - **Deep Learning**
- parte no aprendida:
 - planner clásico

- chequeo geométrico
- MPC

Reinforcement Learning no sería el núcleo del TFM.

Lo dejaría, como mucho, para simulación o trabajo futuro. Hay literatura interesante sobre RL aplicado a planificación óptima en vehículos autónomos, como esta tesis de la UAH sobre planificación óptima de movimiento y aprendizaje por refuerzo en vehículos móviles autónomos, pero para un MVP serio prefiero una arquitectura más controlable y verificable.

3) Pose estimation para aperos: líneas futuras de desarrollo

Este punto **no lo incluiría en el alcance principal del TFM.**

Lo plantearía como una extensión futura.

Qué sería

Sería un módulo de **estimación de pose relativa 6D** entre vehículo y apero para tareas como:

- alineación fina
- maniobras de acople
- docking
- posicionamiento preciso

Por qué no entra en el TFM principal

Porque introduce complejidad adicional en varias dimensiones:

- dataset muy específico
- etiquetado fino

- tolerancias geométricas mucho más exigentes
- integración cerrada con control de maniobra lenta
- mayor dependencia de condiciones visuales y calibración

Cómo lo plantearía como futuro

La línea futura sería:

- detección del apero
- keypoints o segmentación
- estimación de pose 6D
- refinamiento geométrico
- control visual de aproximación

Aquí sí tendría sentido usar:

- **Deep Learning supervisado**
- geometría 3D
- PnP / ICP / visual servoing

Pero no lo metería en el MVP porque desviaría el foco del objetivo principal, que es **tener percepción BEV y planificación híbrida funcionales**.

Stack tecnológico propuesto

Mi propuesta de stack sería la siguiente:

Middleware y sistema

- **ROS 2**

- rviz2 para visualización
- rosbag2 para logging y replay

Percepción y fusión

- **PyTorch**
- **OpenCV**
- **PCL**
- **NumPy**
- **CUDA**

Planificación y control

- **C++** para módulos críticos en tiempo real
- **Python** para experimentación y training
- **MPC** implementado con librerías de optimización tipo CasADi / acados si el entorno lo permite

Simulación y validación

- simulación en **CARLA** o entorno equivalente si aplica
- validación offline con bags
- métricas sobre:
 - ocupación
 - colisiones
 - error lateral
 - éxito de replanificación

- latencia extremo a extremo

Sensores mínimos para el MVP

- 1 LiDAR frontal
 - 2 cámaras RGB
 - GNSS/RTK
 - IMU
 - odometría / CAN
-

Alcance realista del TFM

El entregable que considero realista y sólido sería:

- generar un **BEV útil del entorno**
- extraer de él:
 - ocupación
 - espacio libre
 - obstáculos
- conectar ese BEV con una **planificación híbrida**
- obtener trayectorias locales seguras
- seguirlas con un **MPC**
- validar en simulación y/o dataset grabado

Es decir, el objetivo no sería “autonomía total”, sino demostrar un **pipeline serio de percepción + planificación**.

Resumen corto, directo y defendible ante el tutor

Quiero desarrollar un MVP de conducción autónoma centrado en percepción y planificación.

La percepción se basará en un **Transformer multimodal en BEV** para fusionar cámara, LiDAR y estado del vehículo en una representación cenital del entorno. Esa BEV se usará como entrada a una **planificación híbrida**, donde combinaré planner clásico, validación geométrica y **MPC**, con posible apoyo de aprendizaje supervisado o imitation learning en la capa local.

El objetivo del TFM no será resolver todo el problema de acople de apertos, sino dejar la **pose estimation 6D para una fase futura**, ya que añade mucha complejidad y no es crítica para demostrar el núcleo del sistema.

Mi valoración

Para un TFM, esta propuesta está bien enfocada porque:

- es **técnicamente seria**
- tiene **alcance defendible**
- toca percepción, planificación y control
- permite enseñar ML real sin caer en “end-to-end black box”
- deja una línea futura clara y valiosa

Si quieres, en el siguiente mensaje te lo convierto en cualquiera de estos formatos:

1. **Resumen de 1 minuto para contárselo oralmente al tutor**
 2. **Texto formal de propuesta de TFM**
 3. **Diapositiva única tipo “objetivo + metodología + stack + roadmap”**
-

Cómo se podría titular un TFM de estas características. Que tenga la BEV + optimización de rutas + identificación de cultivos

RouteLLM

Para un TFM que integra **Percepción Multimodal (BEV)**, **Planificación Híbrida (Optimización)** y **Segmentación Semántica (Identificación de Cultivos)**, el título debe reflejar la convergencia entre la robótica móvil avanzada y el sector agrotech.

Aquí tienes varias opciones según el enfoque que quieras enfatizar:

Opción 1: Enfoque Integral (El más académico y descriptivo)

"Arquitectura de Percepción Multimodal en Espacio BEV y Planificación Híbrida de Trayectorias para la Navegación Autónoma en Entornos Agrícolas"

- **Por qué funciona:** Cubre los tres pilares. Define la técnica (multimodal BEV), el entorno (agrícola) y la tarea (navegación/planificación).

Opción 2: Enfoque en Optimización y Eficiencia (Más orientado a Ingeniería de Sistemas)

"Optimización de Rutas Operativas mediante Representaciones Bird's-Eye View y Segmentación Semántica de Cultivos para Maquinaria Autónoma"

- **Por qué funciona:** Resalta la palabra "Optimización", que suele gustar mucho en tribunales de ingeniería, y vincula la identificación de cultivos como la base para esa eficiencia.

Opción 3: Enfoque Vanguardista (Más orientado a Deep Learning / SOTA)

"Sistemas de Navegación Autónoma Basados en Transformers: Fusión Sensorial en BEV y Control Predictivo para la Identificación y Seguimiento de Cultivos"

- **Por qué funciona:** Menciona los "Transformers" y el "Control Predictivo" (MPC), posicionando el trabajo en el estado del arte (State-of-the-Art).

Opción 4: Enfoque de Aplicabilidad (Más orientado a Producto/MVP)

"Diseño e Implementación de un MVP para la Autonomía en Tractores: Integración de Percepción BEV, Selección de Rutas y Clasificación de Cultivos"

- **Por qué funciona:** Es directo y profesional. Al usar "MVP", dejas claro que es un prototipo funcional y serio, alineado con tu perfil de ingeniero industrial y emprendedor.

La recomendación del "IQ 170":

Si buscas el título que mejor defienda la **profundidad técnica** de lo que hemos hablado, yo elegiría:

"Desarrollo de un sistema de navegación autónoma híbrido basado en representaciones Bird's-Eye View y optimización bajo restricciones para la identificación y seguimiento de cultivos"

Desglose del título para tu tutor:

1. **"Sistema de navegación autónoma híbrido"**: Ya le estás diciendo que usas ML + Clásico (punto 2).
2. **"Basado en representaciones Bird's-Eye View"**: Introduces la parte de percepción avanzada y Transformers (punto 1).
3. **"Optimización bajo restricciones"**: Es la forma elegante y técnica de referirse al MPC y a la planificación local/global.
4. **"Identificación y seguimiento de cultivos"**: Cumples con el requisito de segmentación semántica que diferencia este TFM de uno de "un coche en una carretera".

¿Te encaja alguno de estos o prefieres que ajustemos el énfasis en alguna tecnología concreta (p.ej. nombrar explícitamente ROS 2)?

Me quedo con tu propuesta "Desarrollo de un sistema de navegación autónoma híbrido basado en representaciones Bird's-Eye View y optimización bajo restricciones para la identificación y seguimiento de cultivos", pero hazla más corta y concisa

RouteLLM

Para que el título sea más directo y mantenga el rigor técnico exigido en un Máster, podemos eliminar conectores y simplificar la estructura sin perder los tres pilares (BEV, Optimización, Cultivos).

Aquí tienes las 3 mejores opciones concisas, ordenadas de mayor a menor formalidad:

Opción A (La más equilibrada)

"Navegación Autónoma Híbrida en BEV con Optimización de Rutas para Seguimiento de Cultivos"

- **Puntos clave:** Define el paradigma (Híbrida), el espacio de representación (BEV), la técnica de control (Optimización) y la aplicación final (Cultivos).

Opción B (Enfoque en sistemas de percepción-control)

"Arquitectura BEV y Planificación Óptima para la Identificación y Seguimiento de Cultivos"

- **Puntos clave:** Muy directo. Divide el trabajo en "cómo veo el mundo" (BEV) y "cómo me muevo en él" (Planificación Óptima) dentro del contexto agrícola.

Opción C (Máxima brevedad - "Estilo Paper")

"Percepción en BEV y Control Óptimo de Trayectorias en Entornos Agrícolas"

- **Puntos clave:** Es la versión más "limpia". La identificación de cultivos queda implícita dentro de la percepción en entornos agrícolas.

Mi recomendación:

Usa la **Opción A**. Es la que mejor "vende" la complejidad de tu TFM al tutor: dejas claro que no es solo mover un robot, sino que hay una fusión sensorial compleja (BEV), un algoritmo de decisión superior (Optimización) y una tarea de visión específica (Cultivos).

¿Quieres que utilicemos este título para redactar el **objetivo general** y los **objetivos específicos** del TFM?
